AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 2

Attention From First Principles

"The order failed because it was blocked." What does "it" refer to? You knew instantly, without looking at each word in turn. Attention gives a model a way to do something similar, and the arithmetic turns out to fit in a few lines of Java.

Module	AI02 of 13
Lesson	2 of 7
Prerequisites	AI01 L3–L4, AI02 L1
Exam relevance	Out of scope. Explains why long context is costly (supports TS 5.1)
Reading time	~40 minutes

Contents

1. Attention From First Principles

Lesson 2 of 7 · Generative AI and LLM Fundamentals

Attention From First Principles

Key idea

Attention lets every token gather information directly from every other token, weighted by relevance. Each token asks a question (query), offers a label (key) and carries content (value). Comparing queries with keys gives weights, and those weights blend the values. There's no sequential memory to forget and no fixed-size bottleneck. **The price is that comparing everything with everything costs grow with the square of the sequence length.**

> *Before: the seq2seq bottleneck (AI01 L4), softmax (AI01 L3), tokens (AI02 L1). Teaches: embeddings in the model, query/key/value, scaled dot-product attention, self-attention and cross-attention, causal masks, multi-head attention, positional encoding. Used by: AI02 L3 (the full transformer), AI02 L5 (the KV cache), AI02 L6 (position effects).*

The hook: the word that points backward

Your assistant reads this support note:

> "The **order** failed at the warehouse on Tuesday, and the customer's card was fine, because **it** was blocked by a compliance hold."

What does "it" mean? Probably the order, not the card. The evidence sits ten or more words back, past several other nouns.

Architecture	What happens
Bag of words	No order at all, so the reference can't be resolved
RNN	Must have carried "order" through its memory at every step, and may have blurred it
Seq2seq	Everything was squeezed into a single vector
Attention	"it" looks directly at every earlier word and picks what's relevant

Intuition: a database lookup that blends

You already know this structure from HashMaps, with one difference:

	`HashMap.get(query)`	Attention
Match	Exact key or nothing	Every key matches to some degree
Result	One value	A weighted blend of all the values
Learned?	No	The query, key and value representations are all learned

So attention is a **soft lookup**. The query compares itself with every key, getting a similarity for each. Those become weights adding up to 1, and the answer is the weighted sum of the values.

Step 1: Embeddings give tokens meaning

Token IDs are arbitrary integers. The model's first layer is a learned table, with one row per token in the vocabulary.

Token ID	Vector (illustrative)
8123	`[0.12, -0.84, 0.33, ...]`
2201	`[0.91, 0.02, -0.47, ...]`

Each row starts random, and training adjusts it. Vector length is called the **model dimension**, written `d_model`. The original transformer used 512. These are the same idea as word2vec (AI01 L4), trained jointly with the rest of the model.

There's one catch: an embedding still gives "charge" a single vector. Attention is what makes the representation depend on context.

Step 2: Query, Key and value

Each token's embedding is multiplied by three learned matrices, producing three vectors with different jobs.

Vector	The question it answers	Example, for "it"
Query (Q)	What am I looking for?	"I'm a pronoun, so I need a noun that I could refer to"
Key (K)	What do I contain, for matching?	"order": "I'm a singular noun, and the sentence's subject"
Value (V)	What information do I pass on?	"order": its actual meaning

Those explanations are metaphors. Nobody programs "pronoun" or "noun". Training finds whatever matrices lower the loss.

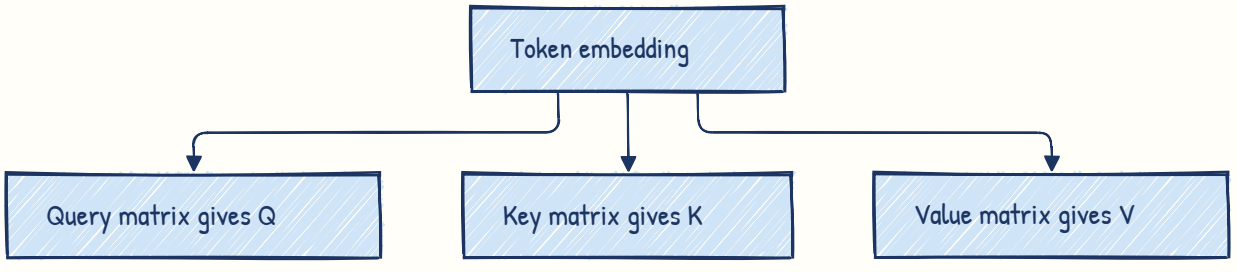


Figure 1. One embedding becomes three vectors, each from its own learned matrix. Training decides what each one comes to represent.

Why three, rather than one? **Because what makes you a good match isn't necessarily what you should pass on.** Separating them gives the model that freedom.

Step 3: The formula

The original paper defines **scaled dot-product attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

Read from the inside out:

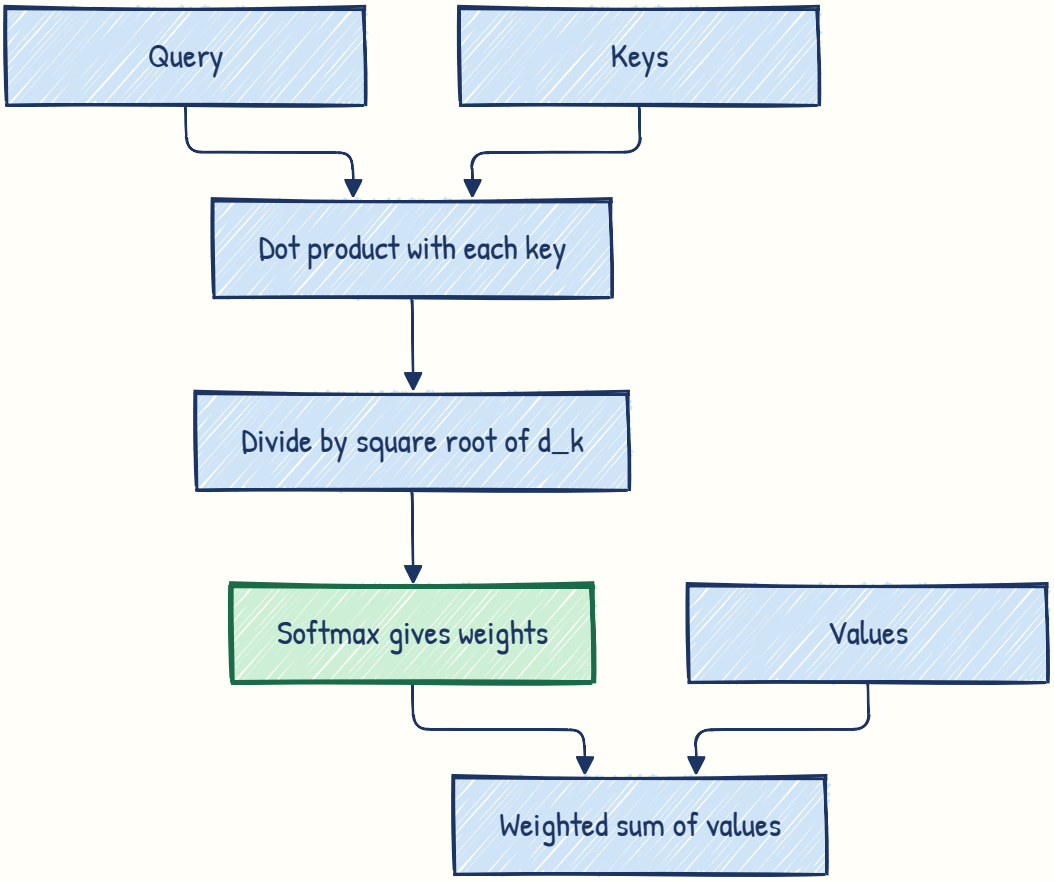


Figure 2. Scaled dot-product attention. Queries and keys produce the weights, and the weights blend the values.

Step	What it does
Dot product	Scores each token's relevance. Similar direction gives a high score
Divide by $\sqrt{d_k}$	Keeps scores in a stable range (explained below)
Softmax	Turns scores into weights adding up to 1
Multiply by V	Produces a new vector blending the values

Here's one row in Java, from `code/ai02-llm/Attention.java`:

```java
static double[] weights(double[] query, double[][] keys, int visibleCount) {
    double[] scores = new double[keys.length];
    double scale = Math.sqrt(query.length);
    for (int i = 0; i < keys.length; i++) {
        scores[i] = i < visibleCount
                ? dot(query, keys[i]) / scale
                : Double.NEGATIVE_INFINITY;
    }
    return softmax(scores);
}
```

For simplicity, this demo uses each token's vector directly as Q, K and V, with hand-written numbers. Real output:

```
tokens = [order, failed, because, it]
'it' attends, no mask: 0.278 0.188 0.181 0.353
'it' output vector:    0.579 0.287 0.128 0.392
'failed' causal mask:  0.389 0.611 0.000 0.000
```

Read the first row: "it" draws 27.8% from "order", more than from "failed" or "because", and 35.3% from itself. Its new vector is a blend, and now **it contains information about the order**.

> *Key idea*
>
> **This is what context-dependent meaning looks like, in numbers.** "it" had one vector before attention, and afterward it has a new one built from its surroundings. Put "charge" next to "card", and it blends differently than next to "battery". That solves the one-vector-per-word weakness from AI01 L4.

Why divide by the square root of d_k?

The paper explains that large dimensions make dot products grow, pushing softmax into regions with very small gradients. Here's the effect, measured on random vectors, by averaging the largest of 8 weights:

```
d=4     top weight unscaled=0.513 scaled=0.336
d=64    top weight unscaled=0.874 scaled=0.358
d=1024  top weight unscaled=0.969 scaled=0.363
```

Without scaling, at high dimension, softmax puts 96.9% on one token, purely by chance. It acts almost like a hard choice, and its gradient nearly vanishes, which is the problem from AI01 L3. With scaling, the top weight stays steady. It's one small division, and training depends on it.

Variations you need to know

Self-attention and cross-attention

Type	Q comes from	K and V come from	Use
Self-attention	This sequence	This same sequence	Tokens understand each other. The core of LLMs
Cross-attention	The decoder	The encoder's output	Translation looks back at the source sentence

Modern LLMs are mostly **decoder-only**, relying on masked self-attention (AI02 L3).

The causal mask

When an LLM trains on next-token prediction, a token must not see the future. Otherwise, it could just copy the answer. So future positions get a score of negative infinity before the softmax, which makes their weight exactly zero. In the output above, "failed" has weights only on "order" and itself, and the last two are `0.000`.

Multi-head attention

One attention pass gives one blend, but a word relates to others in several ways: grammatically, by reference, by topic. So the model runs several attention operations in parallel, called **heads**, and each has its own Q, K and V matrices.

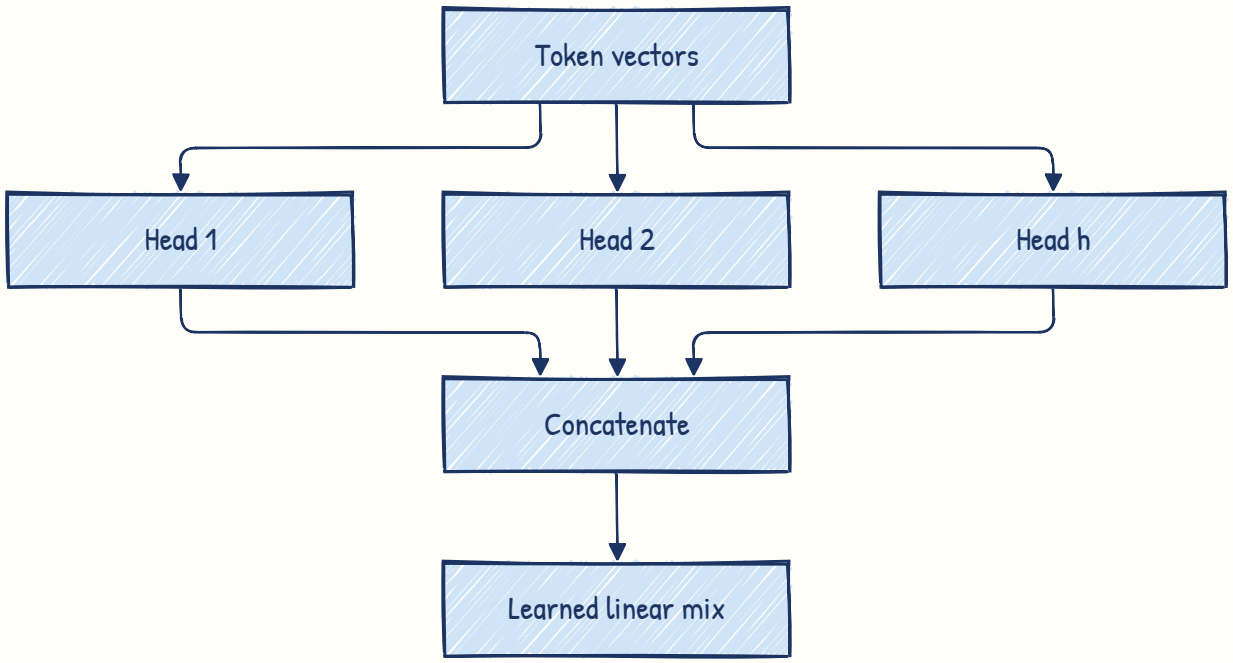


Figure 3. Multi-head attention. The model splits its dimension across heads, runs attention in each, then joins and mixes the results.

The dimension is split, so the total compute stays roughly the same as a single large head:

Model	`d_model`	Heads	Per-head dimension (`d_k`)
Original Transformer (paper)	512	8	64
GPT-2 small	768	12	64

Positional encoding

Attention treats input as a **set**. "Refund denied, not approved" and "Refund approved, not denied" contain the same tokens, and attention alone can't tell them apart. So position information has to be added to each embedding.

Method	How it works	Notes
Sinusoidal	Fixed sine and cosine waves of different frequencies	Original paper. Authors reported near-identical results to learned embeddings
Learned	A table with one vector per position	Simple, but capped at the table's length
RoPE	Rotates Q and K according to relative position	Used in many modern open models

The sinusoidal version gives nearby positions similar vectors. Real output, at dimension 64:

```
cosine(position 0, position 1) = 0.966
cosine(position 0, position 2) = 0.884
cosine(position 0, position 10) = 0.658
cosine(position 0, position 100) = 0.559
```

The cost: quadratic attention

If every token compares itself with every other, comparisons grow with the square of the length.

```
  1,000 tokens: 1,000,000 attention pairs
 10,000 tokens: 100,000,000 attention pairs
100,000 tokens: 10,000,000,000 attention pairs
```

> *Don't over-read these as prices*
>
> Real systems reduce this substantially through caching, batching and hardware (AI02 L5), and API prices aren't quadratic. Prices are generally per token. **The point is why long context has real physical costs, and why providers work so hard on them.**

Why the obvious alternatives fall short

Alternative	What it fixes	What remains broken
A deeper RNN	More capacity	Still sequential, still forgets
Bahdanau attention on an RNN	The bottleneck	Still sequential
CNN over text	Parallelism	Long-range links need many layers
Self-attention	Bottleneck, parallelism and range together	Quadratic cost, and needs position added

How the field solved it

Year	Step	Source
2014	Attention added to RNN translation, to remove the bottleneck	Bahdanau et al.
2017	Attention alone, "dispensing with recurrence and convolutions entirely"	Vaswani et al.
2021	Rotary position embeddings	Su et al.

This wasn't Anthropic's invention. Its contribution here is understanding what's inside trained models, covered in AI02 L6. Its Transformer Circuits work studies these networks directly.

What this means in production

No new architecture is needed for this lesson, and forcing one would add nothing. Attention has direct consequences for the systems you'll design.

Attention fact	Production consequence	Where it's built
Costs rise with context length	Keep prompts lean, and measure latency by input size	AI03 L6, AI10 L5
Everything competes for finite weights	Irrelevant context can distract the model	AI03 L6
Position matters	Put key facts where they'll be noticed	AI02 L6, TS 5.1
The model reads a whole prefix each time	Reusing a prefix is valuable, hence prompt caching	AI03 L5
Instructions and data are mixed in one sequence	Attention can't separate trusted from untrusted text, hence prompt injection	AI09 L2

That last row deserves emphasis:

> *This is the root of prompt injection*
>
> To attention, your system prompt and a retrieved web page are both just token vectors in one sequence. **There's no hardware-level boundary between "instruction" and "data", like the one between SQL and bound parameters.** AI09 L2 builds defenses from this fact.

Java: it's only arithmetic

You'd never serve a transformer with this code, because real systems use optimized tensor libraries on GPUs. But it's valuable to see that the mechanism isn't magic: loops, dot products and a softmax.

```java
static double[] mix(double[] weights, double[][] values) {
    double[] out = new double[values[0].length];
    for (int i = 0; i < values.length; i++) {
        for (int d = 0; d < out.length; d++) {
            out[d] += weights[i] * values[i][d];
        }
    }
    return out;
}
```

Trade-offs

Design choice	Gain	Cost
Attention instead of recurrence	Parallel training, direct long-range links	Quadratic growth, needs position
More heads	More kinds of relationship	Less dimension for each head
Larger `d_model`	More capacity	Memory and compute
Learned positions	Simple	Capped at a fixed length

When not to think about any of this: day to day, you call an API. This knowledge pays off when reasoning about long context, cost, position effects and injection.

Common mistakes

1. **Reading attention weights as an explanation.** A high weight shows where information flowed in one head, and doesn't prove why the model decided something.
2. **Thinking Q, K and V are hand-designed.** They're learned, and their "meanings" are analogies.
3. **Forgetting position.** Without it, attention can't distinguish word order.
4. **Assuming more context is free.** It has costs in compute, money and focus.
5. **Believing a delimiter gives security.** To the model, it's only more tokens.

What to say in an interview

For "explain attention", use the lookup analogy: "It's a soft HashMap. Each token's query is compared with all keys, softmax turns scores into weights, and the output is a weighted blend of values. That makes representations context-dependent, and removes RNNs' bottleneck. The cost is quadratic." Then add scaling and masking if they probe further.

Certification lens

How this lesson maps to CCAR-F

Not tested. Claude's internal architecture is out of scope. What carries over is the practical effect, tested in TS 5.1: place key findings first, and use clear section headers.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Sequence order · Medium. Put these in order for computing one token's attention output: (1) weighted sum of values, (2) softmax, (3) dot product of query and keys, (4) divide by the square root of d_k.

A **3, 4, 2, 1.** Score, scale, turn into weights, then blend.

Q Q2 · Calculation · Medium. A model has d_model = 1024 and 16 heads. What's d_k? How many attention pairs are there for 4,000 tokens, before any mask?

A d_k = 1024 / 16 = **64**. Pairs = 4,000² = **16,000,000**.

Q Q3 · Assertion and reasoning · Hard. Assertion: wrapping retrieved documents in XML tags guarantees that the model won't follow instructions inside them. Reason: attention treats all tokens as entries in the same sequence. Choose: (A) both true, and R explains A; (B) both true, and R doesn't explain A; (C) A true, R false; (D) A false, R true.

A **D.** R is true, and it's exactly why A is false. Tags help the model understand structure, but they aren't an enforced boundary. Defenses have to include limits outside the model (AI09 L2).

Revision - Lesson 2

→ Attention is a soft lookup: query compared with keys gives weights, and weights blend values.

→ Formula: softmax(Q K^T / √d_k) V.

→ Q, K and V come from learned matrices. Their "meanings" are analogies.

→ Dividing by √d_k stops softmax saturating at high dimension. Unscaled, 96.9% went to one token in the demo.

→ Attention makes each token's vector depend on context.

→ A causal mask stops tokens seeing the future. Future weights are exactly zero.

→ Multi-head runs parallel attentions, splitting the dimension (512 / 8 = 64 in the original).

→ Attention alone ignores order, so positions are added: sinusoidal, learned or RoPE.

→ Comparisons grow with the square of length.

→ Instructions and data share one sequence, which is the root of prompt injection.

Your notes